Defining a DAG in Python

In this train, we'll guide you through the process of using Python to define a DAG in Apache Airflow, covering the essential code structure and syntax.

Learning objectives

> - Know the process of using Python to define a DAG.

Representing DAGs as Python scripts

Most well-known workflow management tools make use of a DAG to represent the flow of data and corresponding tasks within a pipeline. Depending on the management system, how these DAGs are defined can vary. For instance, Azkaban and [Argo](https://argoproj.github.io/) both use `YAML` configuration files, while Conductor uses a `JSON` format, and [Apache NiFi](https://nifi.apache.org/) uses an interactive UI. Other workflow tools opt to implement DAGs directly within code instead of within a templating language. This choice provides more flexibility when building workflows, but can come at a cost of the resulting DAGs being more difficult to read or debug. Examples of frameworks using code-based DAGs include Luigi and Airflow, which both utilise Python scripts to define pipelines. Each DAG definition flavour, whether a scripting language, UI, or text-based method, has its pros and cons. However, because the focus of this train is Airflow, we will focus on defining DAGs within Python scripts. As shown in Figure 1, in Airflow each data pipeline is represented by a single DAG file in the form of a Python script (`.py` file). Due to its intention to operate at scale, there is no limit to the number of these pipeline definitions or scripts that can be hosted within an Airflow server.

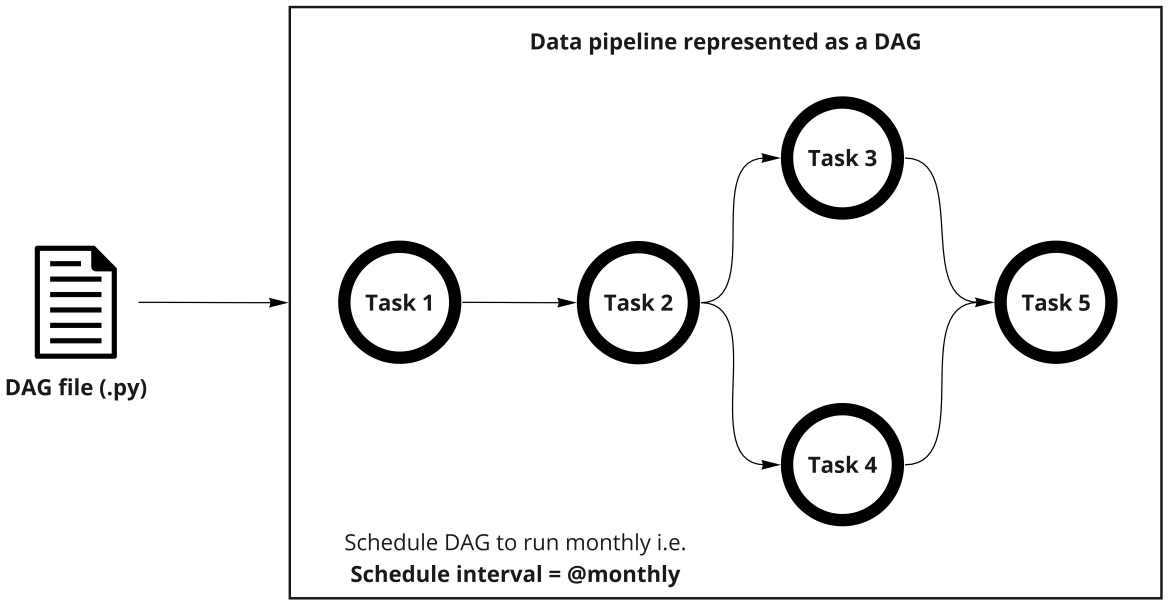


Figure 1: Defining a DAG as a Python script.

Conclusion

Defining DAGs in Python allows for flexibility, scalability, and automation in workflow management. By leveraging Apache Airflow, you can design efficient and maintainable data pipelines that orchestrate tasks seamlessly. Understanding how to structure DAGs in Python is fundamental to building robust workflows that handle complex dependencies, ensuring reliable data processing.

Key Takeaways

- **Definition of a DAG** - What a DAG is and how it structures workflows.
- **Airflow and Python** - How Python is used to define DAGs in Airflow
- **Airflow vs Others** - Comparison of Airflow with other Workflow Tools.
- **Scalability** - Defining DAGs in Python enables scalable pipeline design.